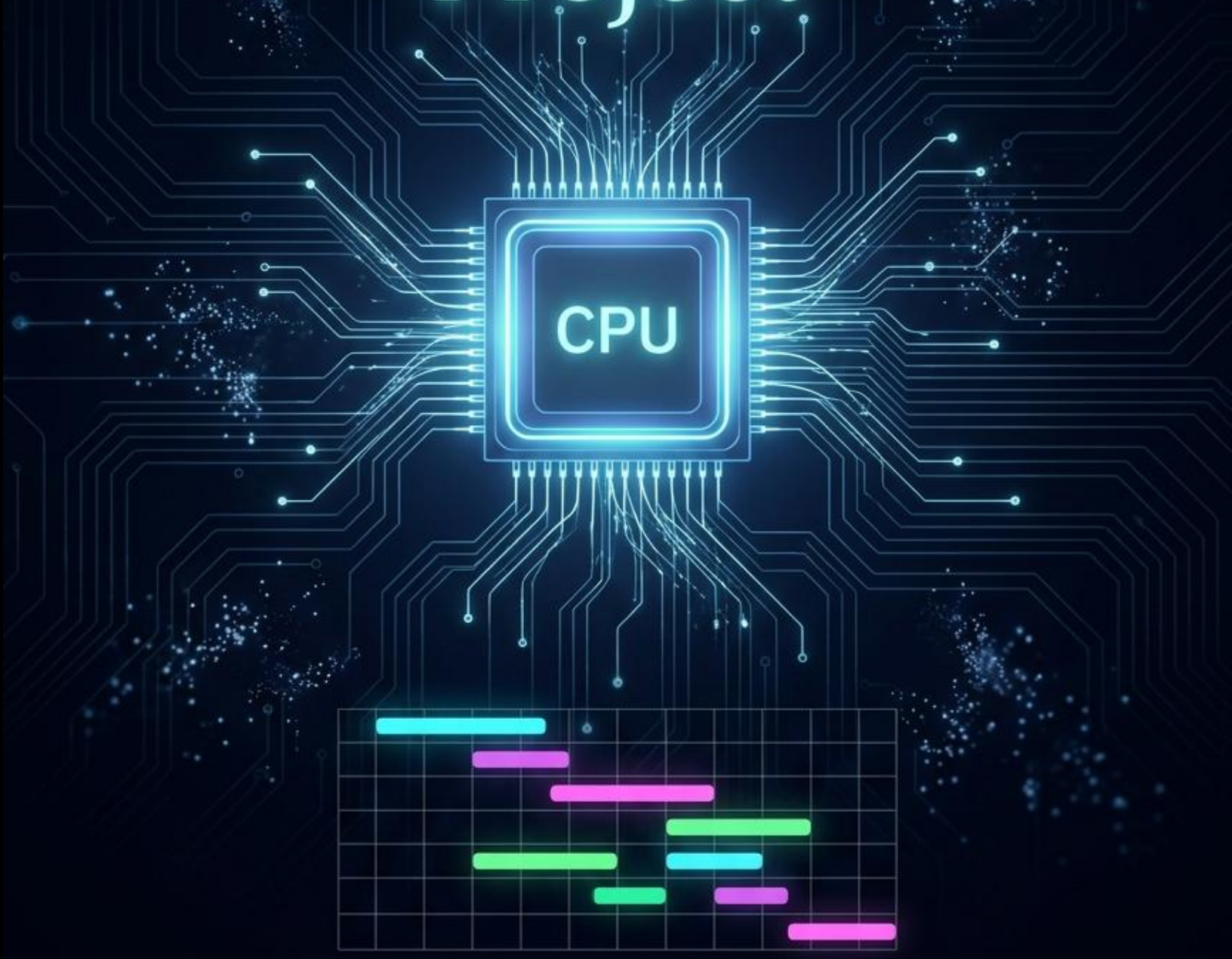
CSE335s (UG2023) - Operating Systems

CPU Scheduler Project



Team Members

Name	ID
Ahmed Mohamed Hassan Elmarsafy	2300099
Mohammed Zaki Abdelazim Elsayed	2300465
Ahmed Sameh Mohamed Hassouna	2300082
Mohamed Osama Fathy	2300533
Hassan Ebrahim Mohamed	2300166
Omar Wael Mahmoud Mahmoud	2300109
Ahmed Abdelaziz Mohamed Omar	2300112

Table of Contents

Team Members 1

Introduction 3

- **1. First-Come, First-Served (FCFS) Scheduling** 4
- **2. Shortest Job First (SJF) Scheduling** 4
  - **2.1 Preemptive** 4
  - **2.2 Non-Preemptive** 4
- **3. Priority Scheduling** 5
- **4. Round Robin (RR) Scheduling** 5

Project Overview 6

- **Model 1: Static Mode** 6
- **Model 2: Live Simulation Mode** 7

GUI 7

Scheduling Algorithms Analysis and Testing 8

- **1. FCFS** 8
  - **EX_1** 8
  - **EX_2** 9
  - **EX_3** 10
- **2.1 SJF - Preemptive** 11
  - **EX_1** 11
  - **EX_2 – Dynamic** 12
- **2.2 SJF - Non-Preemptive** 13
  - **EX_1** 13
  - **EX_2** 14
- **3.1 Priority – Preemptive** 15
  - **EX_1** 15
  - **EX_2 – Dynamic** 16
- **3.2 Priority – Non-Preemptive** 17
  - **EX_1** 17
- **4. Round Robin** 18
  - **EX_1** 18
  - **EX_2** 19
  - **EX_3 – Dynamic** 20

Link to exe file 21

Introduction

In a modern operating system, multiple processes often exist simultaneously, each competing for the CPU to execute their instructions. Since a CPU can typically execute only one process at a time, the operating system must decide which process gets access to the CPU and for how long. This decision-making mechanism is known as CPU Scheduling.

CPU Scheduling is one of the fundamental functions of an operating system. It plays a critical role in determining the overall performance, responsiveness, and efficiency of the system. A well-designed scheduling algorithm ensures that the CPU is utilized to its maximum potential while maintaining fairness among all running processes.

The primary goal of CPU scheduling is to maximize CPU utilization, minimize waiting time, and improve the overall throughput of the system. Various scheduling algorithms have been developed over the years, each with its own strategy, advantages, and trade-offs. The choice of scheduling algorithm significantly impacts system performance metrics such as:

Burst Time	The total time required by a process to complete its execution on the CPU.
Waiting Time	The total time a process spends waiting in the ready queue before it gets CPU access.
Turnaround Time	The total time taken from process submission to process completion, including both execution and waiting time.

This project implements and simulates four CPU scheduling algorithms:
- First Come First Served (FCFS)
- Shortest Job First (SJF)
- Priority Scheduling
- Round Robin (RR)

1. First-Come, First-Served (FCFS) Scheduling

Definition:

- The process that arrives **first** gets executed first.
- Operates like a queue (FIFO – First In, First Out).

Key Features:

- Simple and easy to implement.
- Non-preemptive (once a process starts, it runs till completion).
- Can cause **convoy effect** (long processes delay shorter ones).
- Poor response time for interactive systems.

---

2. Shortest Job First (SJF) Scheduling

2.1 Preemptive

Definition:

- The process with the **shortest remaining burst time** is executed first.
- If a new process arrives with a **shorter burst time**, it **preempts** the running process.

Key Features:

- Preemptive – **minimizes turnaround time**.
- Ensures short jobs get quick execution.
- **High context switching overhead.**
- Starvation risk for longer processes.

————————————————————————————————

2.2 Non-Preemptive

Definition:

- The process with the **shortest burst time** is executed first.

Key Features:

- **Minimizes average waiting time**.
- Non-preemptive – once a process starts, it completes execution.
- **Starvation** risk – long processes may never get CPU time.

3. Priority Scheduling

Definition:

- Each process is assigned a **priority** (lower number = higher priority).
- The CPU executes the **highest-priority** process first.

Key Features:

- Can be **preemptive** (interrupts lower-priority processes) or **non-preemptive**.
- Suitable for **real-time systems** where urgent tasks must run first.
- **Starvation** risk – low-priority processes may never execute.
- Can be improved using **aging** (gradually increasing priority of waiting processes).

---

4. Round Robin (RR) Scheduling

Definition:

- Each process gets a fixed time quantum.
- If a process does not complete within its assigned time, it is **preempted** and placed at the end of the queue.
- If a process requires less time than the allocated quantum, it finishes execution without using the full time slice.
- If there are n processes in the ready queue and the time quantum is q, then No process waits more than $(n-1)q$ time units.
- Quantum value determines the behavior of RR:
  - q large → FIFO
  - q small → too much overhead
  - q must be large with respect to context switch, otherwise overhead is too high.

Key Features:

- Ensures **fairness** – all processes get CPU time.
- Ideal for **time-sharing systems** .
- Performance depends on the **time quantum** chosen.
- More **context switching overhead** than FCFS or SJF.

Project Overview

This project is a CPU Scheduling Simulator — a desktop GUI application that visualizes and simulates the execution of four classic CPU scheduling algorithms. The system is designed to help users understand how different scheduling strategies affect process execution order, waiting time, turnaround time, and overall CPU performance in operating systems.

The simulator provides an interactive environment where users can define processes with their required attributes, select a scheduling algorithm, and observe how the CPU handles process execution. The results are displayed using a dynamic Gantt Chart, a continuously updating burst time table, and calculated performance metrics including average waiting time and average turnaround time.

The application supports two different operating modes: a static execution mode and a real-time live simulation mode, allowing users to either analyze results instantly or visualize the scheduling process step by step.

Model 1: Static Mode

In this mode, all processes are entered at the beginning of the simulation (at time = 0).

Idea:

- The user inputs all processes at once (Arrival Time, Burst Time, Priority, etc., depending on the scheduling algorithm).
- The system starts the scheduling immediately without any changes during execution.
- The entire scheduling is computed in a single step.
- A final Gantt Chart is generated after completing all calculations.

Characteristics:

- No interaction occurs during execution.
- No processes can be added or modified while the simulation is running.
  Time is not visually simulated or animated.
- This mode is suitable for quick analysis and obtaining final results in a direct and simple way.

Model 2: Live Simulation Mode

This mode simulates CPU scheduling in real time with continuous system interaction.

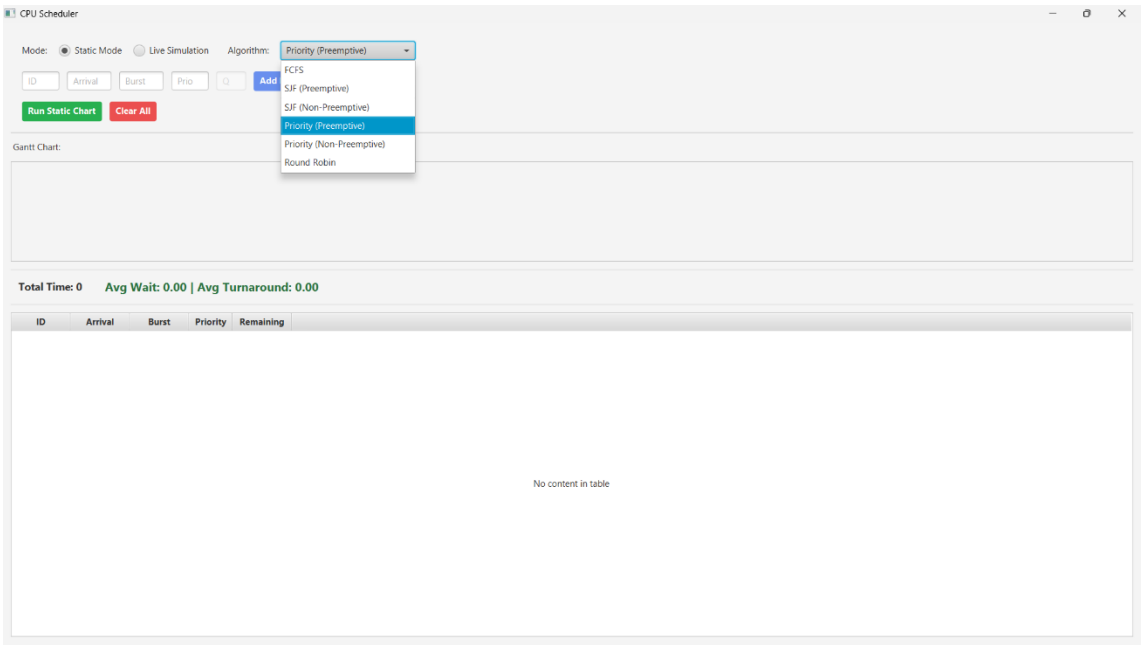
Idea:

- A system clock (time) runs continuously, representing real-time execution (1 unit of time = 1 second).
- The Gantt Chart is updated dynamically as time progresses.
- A Pause/Resume button is provided to control the execution flow.

Characteristics:

- The simulation can be paused at any moment during execution.
- While the simulation is paused:
  - New processes can be added dynamically.
  - The arrival time of newly added processes is automatically set to the current system time at the moment of pause.
- After resuming, the scheduler continues execution from the paused state, and the Gantt Chart updates in real time accordingly.
- This mode closely mimics the behavior of a real CPU operating system with dynamic process handling and time-based execution.

GUI



Scheduling Algorithms Analysis and Testing

1. FCFS

EX_1

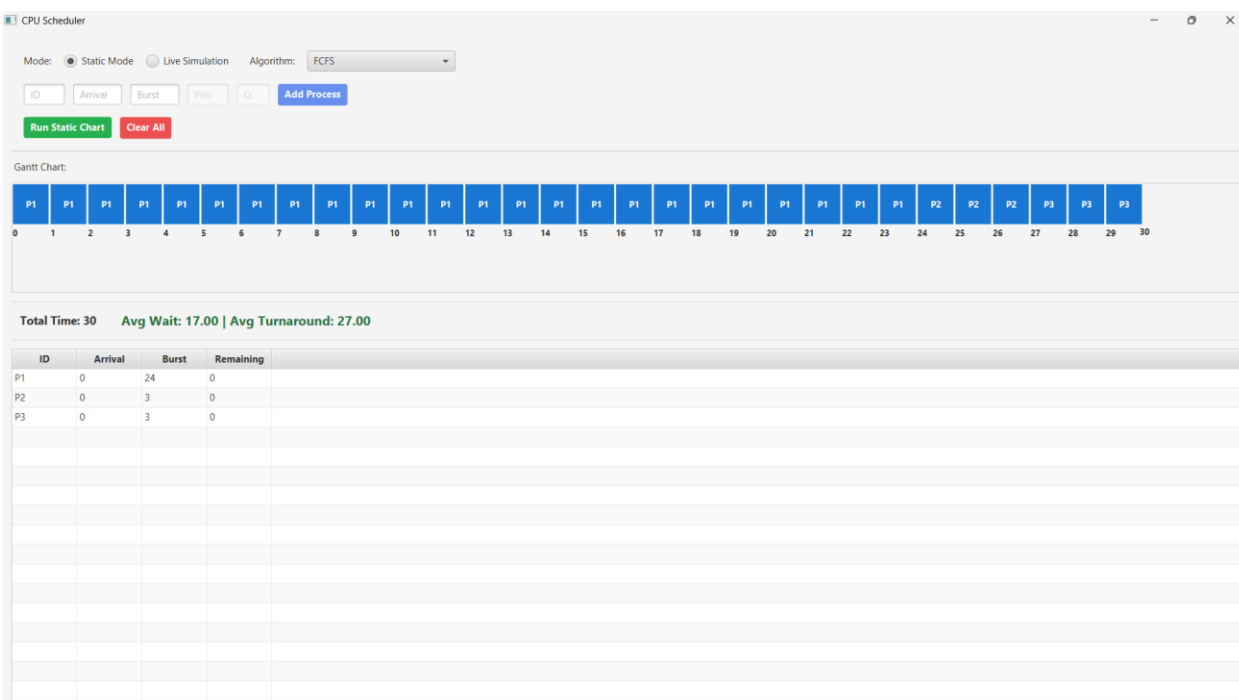
- **Note:** This example is taken from the lecture slides.
- **Static Mode**

Process	Burst Time
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
  The Gantt Chart for the schedule is:

- Waiting time for P_1 = 0; P_2 = 24; P_3 = 27
- Average waiting time: (0 + 24 + 27)/3 = 17

Simulation Results:



EX_2

- **Note:** This example is taken from the lecture slides.
- **Static Mode**

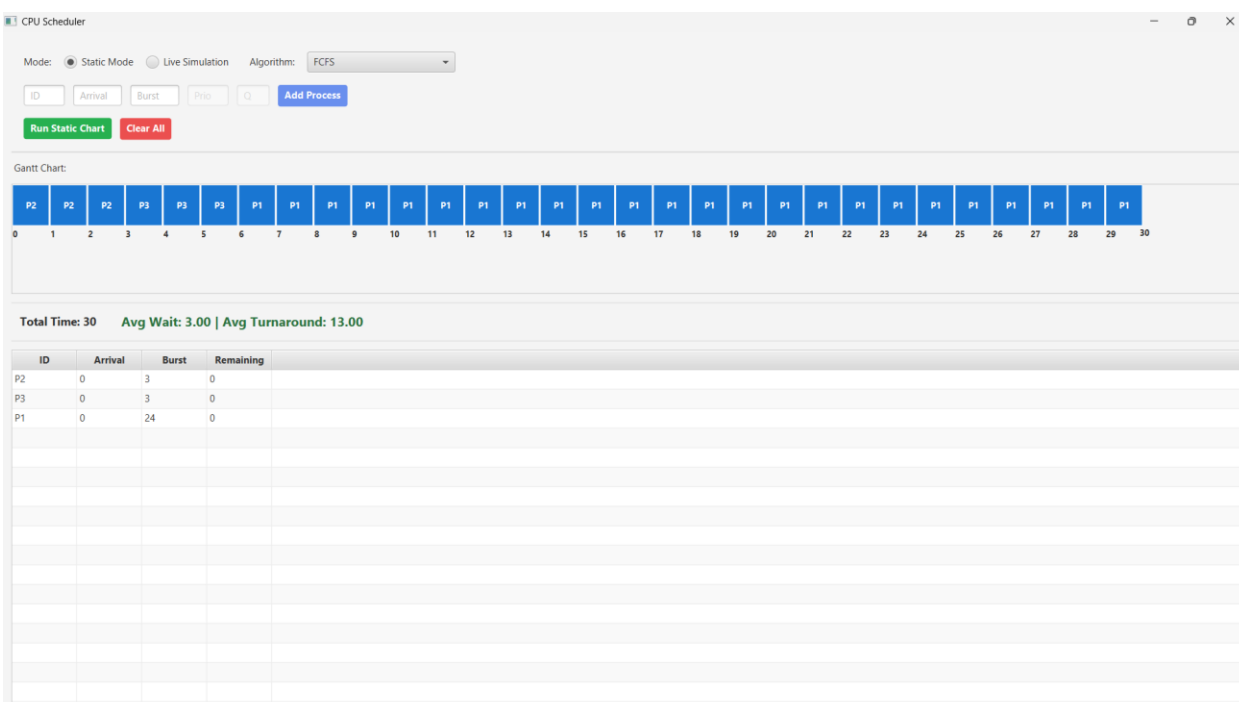
Suppose that the processes arrive in the order:

$$P_2 , P_3 , P_1$$

- The Gantt chart for the schedule is:

- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$

Simulation Results:



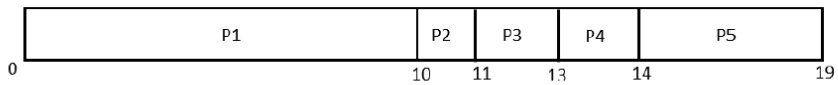
EX_3

- **Note:** This example is taken from Sheet 5.
- **Static Mode**

Consider the following set of processes, with length of CPU burst given in milliseconds.
The processes are assumed to have arrived in the order P1, P2, P3, P4, P5 all at time 0.

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

FCFS:



Turnaround time:

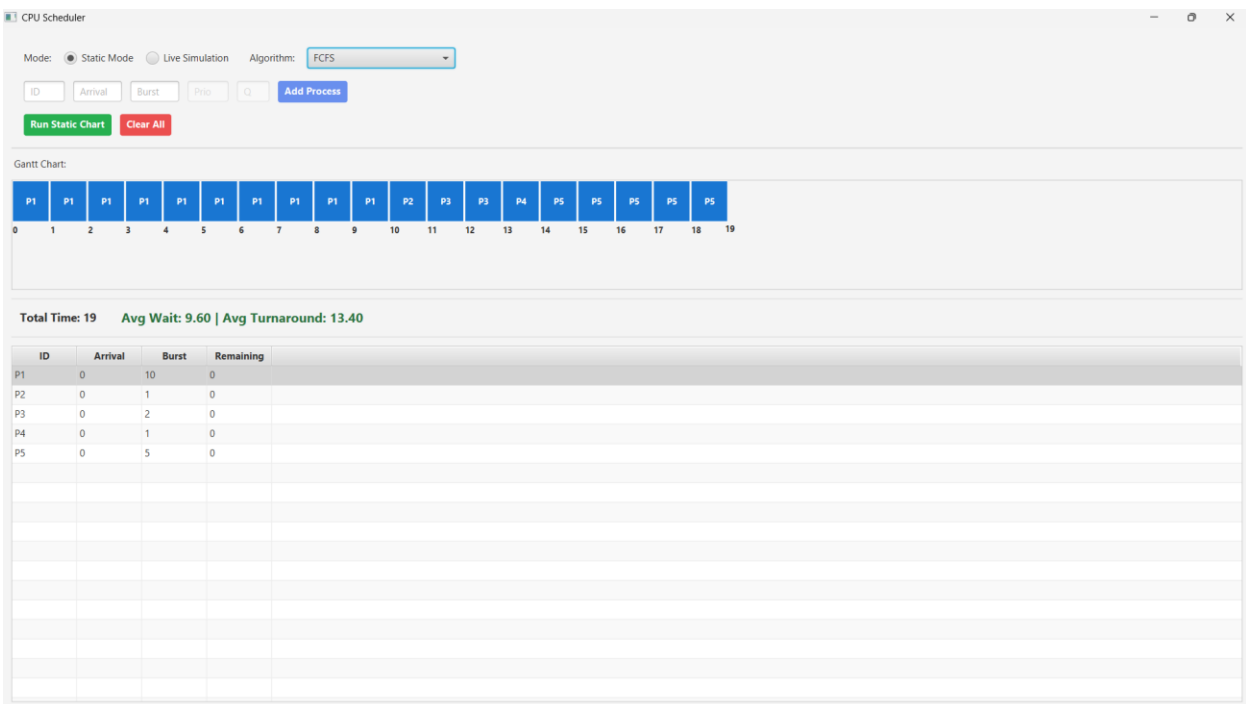
- P1: (10 – 0) = 10 ms
- P2: (11 – 0) = 11 ms
- P3: (13 – 0) = 13 ms
- P4: (14 – 0) = 14 ms
- P5: (19 – 0) = 19 ms

Waiting time:

- P1: (10 – 10) = 0 ms
- P2: (11 – 1) = 10 ms
- P3: (13 – 2) = 11 ms
- P4: (14 – 1) = 13 ms
- P5: (19 – 5) = 14 ms

Average = (0 + 10 + 11 + 13 + 14) / 5 = 9.6 ms

Simulation Results:



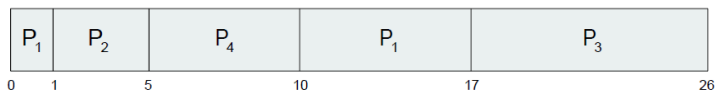
2.1 SJF - Preemptive

EX_1

- **Note:** This example is taken from the lecture slides.
- **Static Mode**

Process	*Arrival* Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

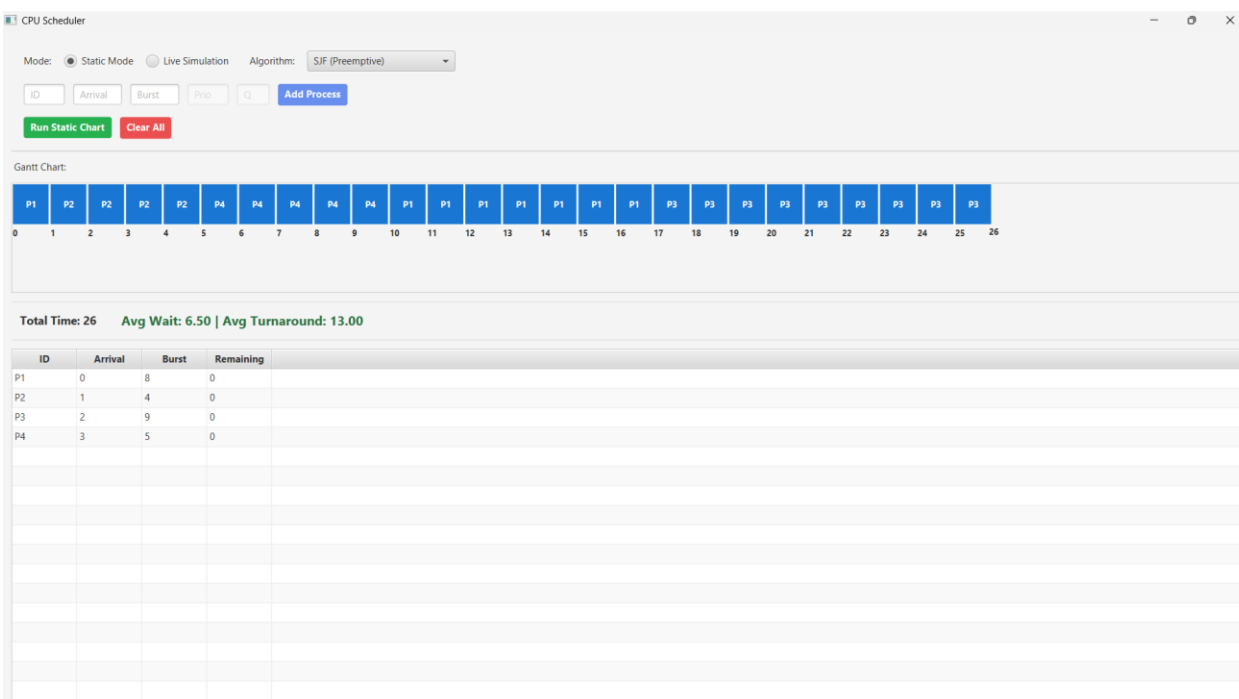
- *Preemptive* SJF Gantt Chart



- Average waiting time = [(10-1)+(1-1)+(17-2)+(5-3)]/4 = 26/4 = 6.5 msec

---

Simulation Results:

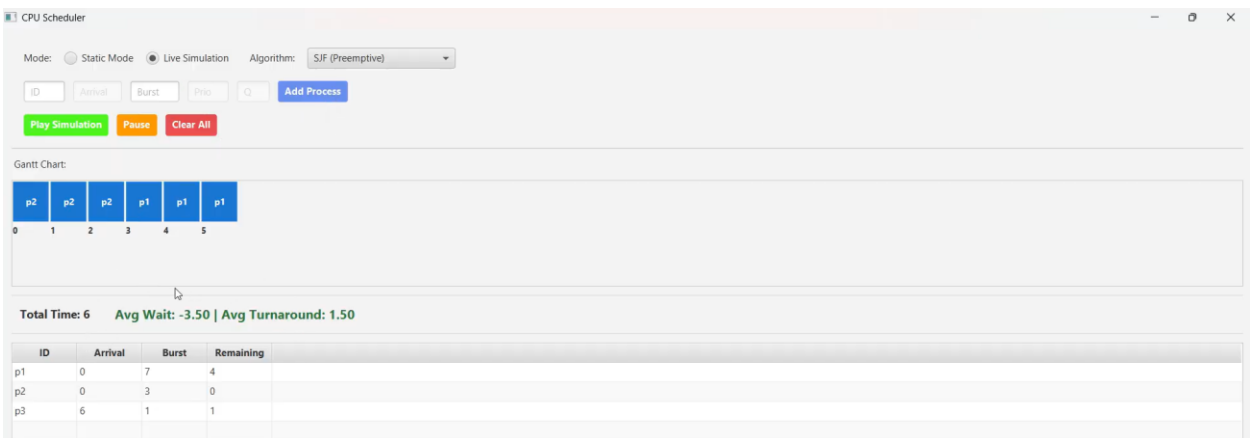


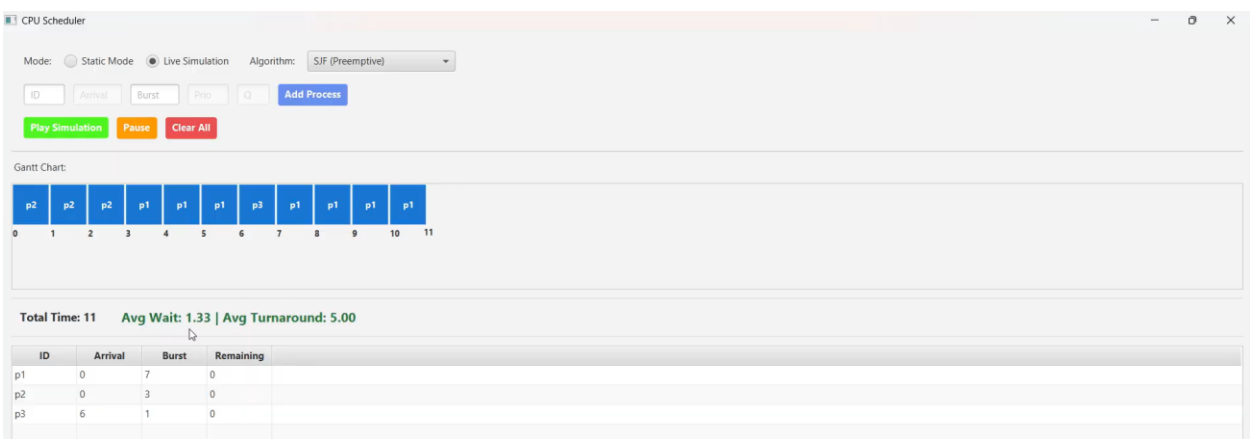
EX_2 - Dynamic

➢ Live Simulation Mode

1. Initially, processes P1 and P2 were entered before starting the simulation, both with arrival time = 0.

2. The simulation was then paused, a new process P3 was added, and the simulation was resumed.





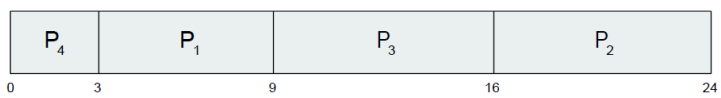
2.2 SJF - Non-Preemptive

EX_1

- **Note:** This example is taken from the lecture slides.
- **Static Mode**

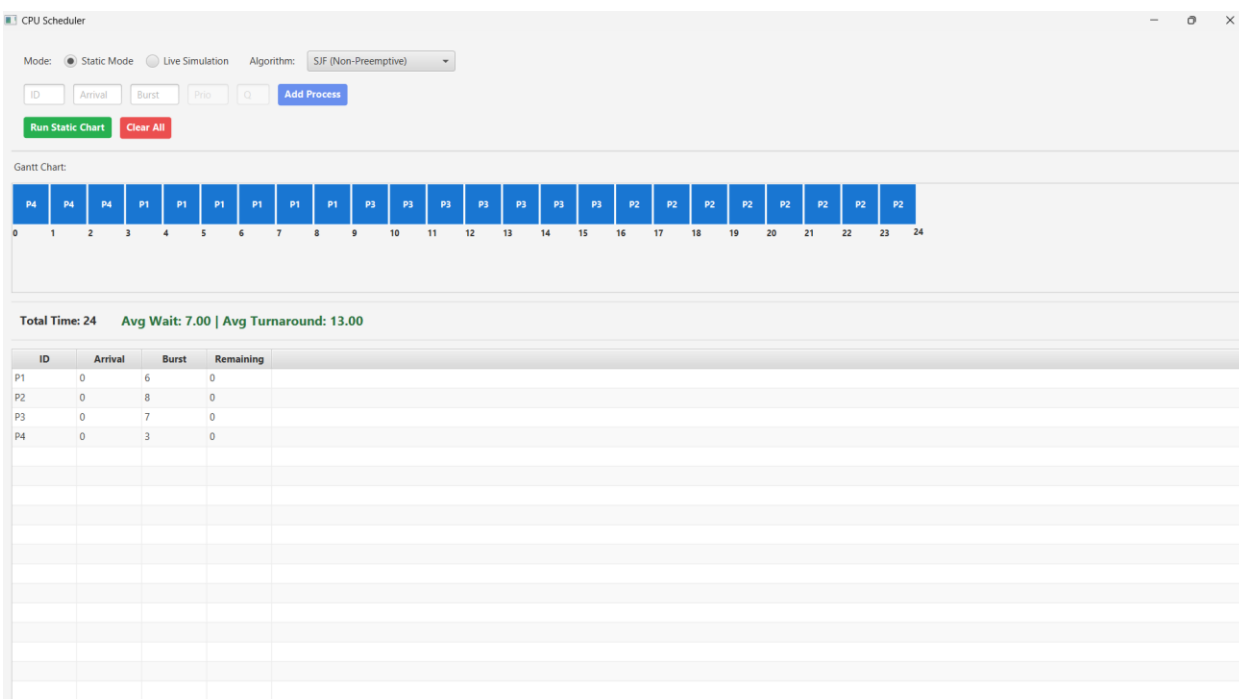
Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

- Nonpreemptive SJF scheduling chart



- Average waiting time = (3 + 16 + 9 + 0) / 4 = 7

Simulation Results:



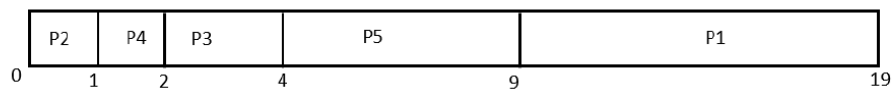
EX_2

- **Note:** This example is taken from Sheet 5.
- **Static Mode**

Consider the following set of processes, with length of CPU burst given in milliseconds. The processes are assumed to have arrived in the order P1, P2, P3, P4, P5 all at time 0.

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

SJF:



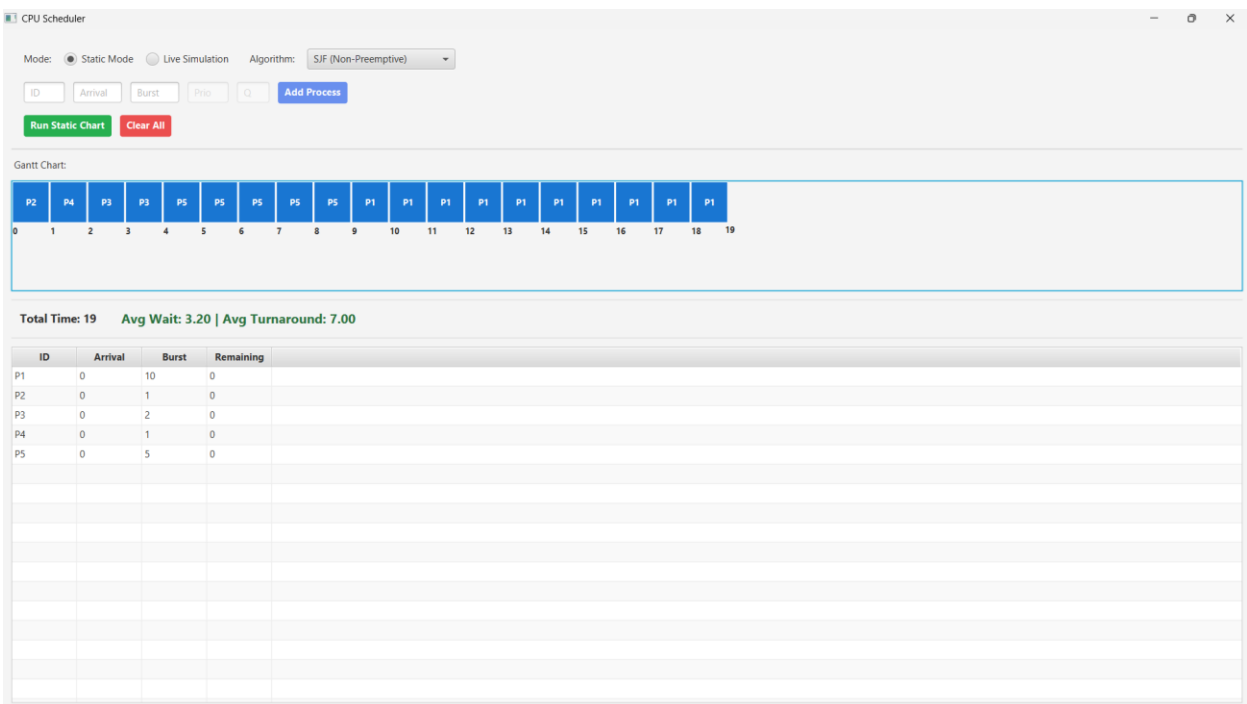
Turnaround time:

- P1: (19 – 0) = 19 ms
- P2: (1 – 0) = 1 ms
- P3: (4 – 0) = 4 ms
- P4: (2 – 0) = 2 ms
- P5: (9 – 0) = 9 ms

Waiting time:

- P1: (19 – 10) = 9 ms
- P2: (1 – 1) = 0 ms
- P3: (4 – 2) = 2 ms
- P4: (2 – 1) = 1 ms
- P5: (9 – 5) = 4 ms

Simulation Results:



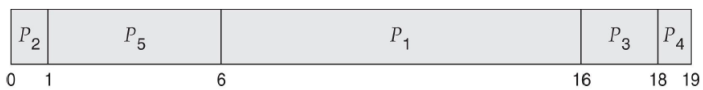
3.1 Priority – Preemptive

EX_1

- **Note:** This example is taken from the lecture slides.
- **Static Mode**

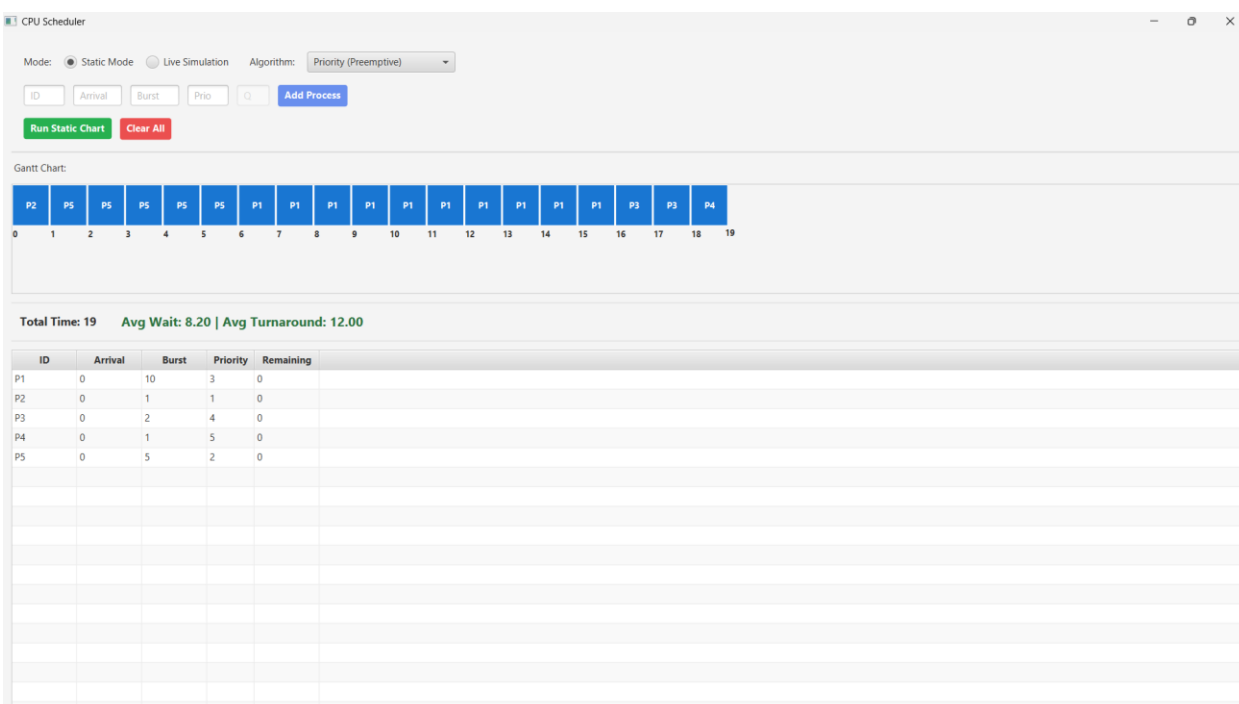
Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

- Priority scheduling Gantt Chart



- Average waiting time = 8.2 msec

Simulation Results:



EX_2 - Dynamic

➢ Live Simulation Mode

1. Initially, processes P1 was entered before starting the simulation with arrival time = 0.

2. The simulation was then paused, a new process P2 was added, and the simulation was resumed.

3.2 Priority – Non-Preemptive

EX_1

- **Note:** This example is taken from Sheet 5.
- **Static Mode**

Consider the following set of processes, with length of CPU burst given in milliseconds. The processes are assumed to have arrived in the order P1, P2, P3, P4, P5 all at time 0.

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

Non-preemptive priority (a smaller priority number implies a higher priority):

P2	P5	P1	P3	P4

0 1 6 16 18 19

Turnaround time:

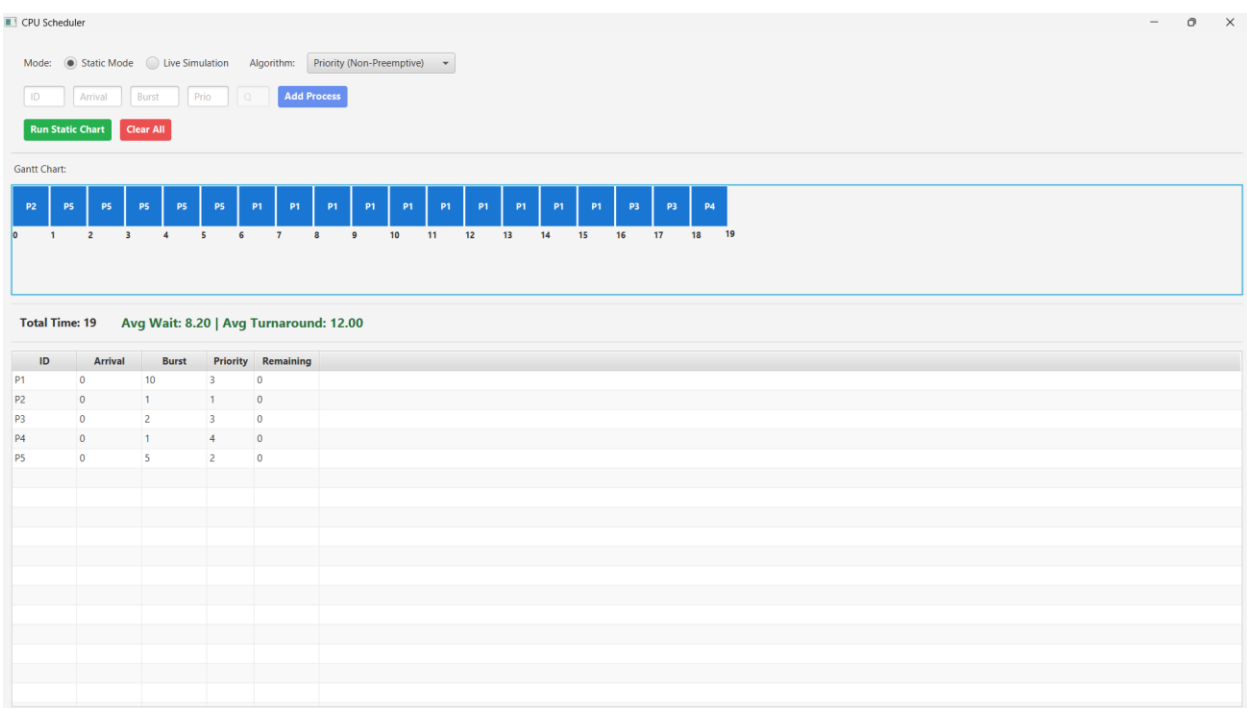
- P1: (16 – 0) = 16 ms
- P2: (1 – 0) = 1 ms
- P3: (18 – 0) = 18 ms
- P4: (19 – 0) = 19 ms
- P5: (6 – 0) = 6 ms

Waiting time:

- P1: (16 – 10) = 6 ms
- P2: (1 – 1) = 0 ms
- P3: (18 – 2) = 16 ms
- P4: (19 – 1) = 18 ms
- P5: (6 – 5) = 1 ms

Average = (6 + 0 + 16 + 18 + 1) / 5 = 8.2 ms

Simulation Results:



4. Round Robin

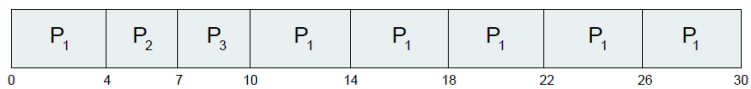
EX_1

- **Note:** This example is taken from the lecture slides.
- **Static Mode**

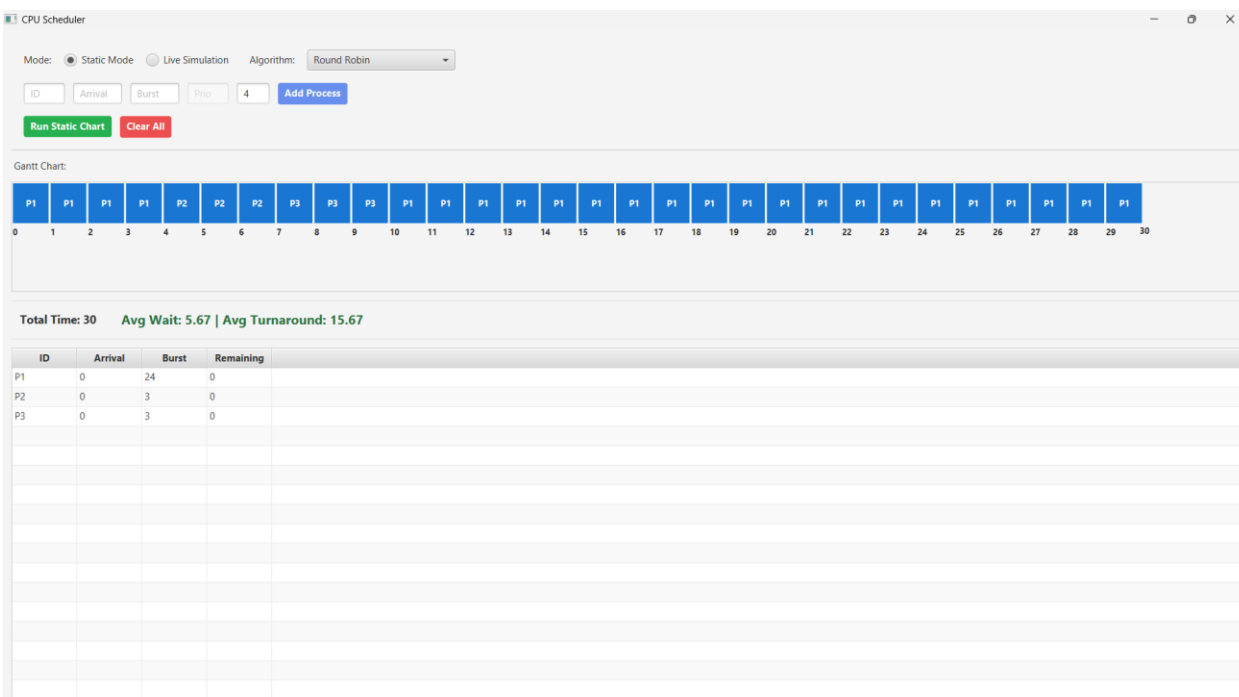
RR with Time Quantum = 4

Process	Burst Time
P_1	24
P_2	3
P_3	3

- The Gantt chart is:



Simulation Results:



EX_2

- **Note:** This example is taken from Sheet 5.
- **Static Mode**

Consider the following set of processes, with length of CPU burst given in milliseconds. The processes are assumed to have arrived in the order P1, P2, P3, P4, P5 all at time 0.

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

RR (quantum=1):

P1	P2	P3	P4	P5	P1	P3	P5	P1	P5	P1	P5	P1	P5	P1	P1	P1	P1	P1

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19

Turnaround time:

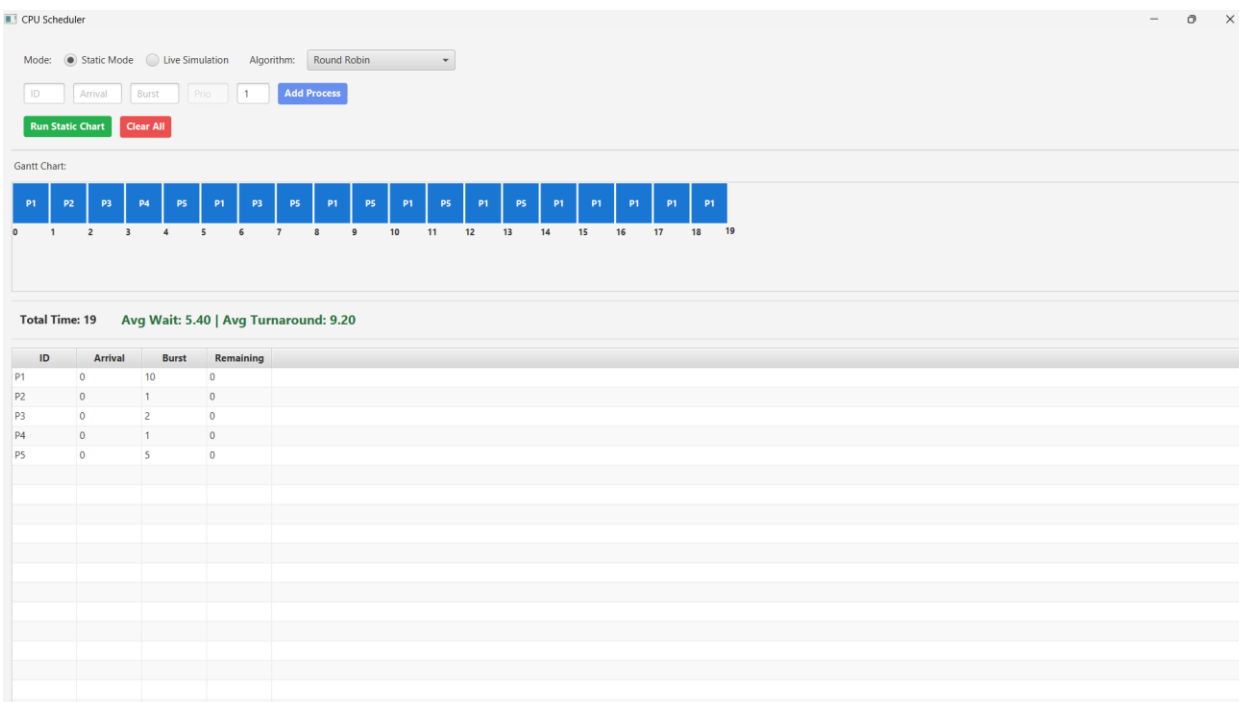
- P1: (19 – 0) = 19 ms
- P2: (2 – 0) = 2 ms
- P3: (7 – 0) = 7 ms
- P4: (4 – 0) = 4 ms
- P5: (14 – 0) = 14 ms

Waiting time:

- P1: (19 – 10) = 9 ms
- P2: (2 – 1) = 1 ms
- P3: (7 – 2) = 5 ms
- P4: (4 – 1) = 3 ms
- P5: (14 – 5) = 9 ms

Average = (9 + 1 + 5 + 3 + 9) / 5 = 5.4 ms

Simulation Results:



EX_3 - Dynamic

➢ Live Simulation Mode

1. Initially, processes P1 and P2 were entered before starting the simulation, both with arrival time = 0.

2. The simulation was then paused, a new process P3 was added, and the simulation was resumed.

Link to exe file

This Google Drive link contains:

- exe file
- (SJF_Preemptive + Priority) - Dynamic Example
- (SJF_Non-Preemptive + RR) - Dynamic Example

Main Drive Link:
https://drive.google.com/drive/folders/1l0yn5xbzmHOUyp6gX9xIPCoMst41UUgJ?usp=drive_link

---

Separate links:

🔗 **exe file:**

https://drive.google.com/file/d/1XDMg1b9wHc6ktiGGLE2s7Y6ljRu7T7tQ/view?usp=drive_link

🔗 **(SJF_Preemptive + Priority) - Dynamic Example:**

https://drive.google.com/file/d/1R0Z8PPJqr6ZteCVBq5nJZbf3P7KHk4cd/view?usp=drive_link

🔗 **(SJF_Non-Preemptive + RR) - Dynamic Example:**

https://drive.google.com/file/d/1lhc1eVW415UppIPMzOEIb1-KpG5XN9oY/view?usp=drive_link